



PARTICIPANT LAB GUIDE · DATA ENGINEERING PROGRAM

Hands-On Lab Guide

Eighteen step-by-step labs spanning the full program — PySpark Fundamentals, Databricks & Azure, Azure Databricks Workflows, and Spark Performance — all built on one shared **SkyOps Analytics** fleet dataset you set up once and reuse throughout.

18

Labs

4

Tracks

1

Dataset



Step by step

What is inside

Work through the tracks in order — do **Setup** first, then the labs that match your course. Each lab is self-contained with a goal, prerequisites, numbered steps, an expected result and a checkpoint. Tick the boxes as you go.

0	Setup & the SkyOps Dataset	→
Environment options & the one-time data bootstrap every lab uses		
A	PySpark Fundamentals	→
7 labs · sessions, SQL/DataFrame parity, windows, streaming, end-to-end		
B	Databricks & Azure	→
4 labs · collaboration & Git, architecture, jobs, Pandas analysis		
C	Azure Databricks Workflows	→
1 lab · deploy a workload via UI and as a bundle		
D	Spark Performance	→
6 labs · plans, the Spark UI, caching, joins, memory, UDFs		

Need an environment? Local PySpark covers Tracks A and D; a free Databricks workspace covers Tracks B and C. Full instructions are in Setup section 0.2.



Setup & the SkyOps Dataset

Get a working Spark environment and load the shared SkyOps Analytics data that every lab in this guide builds on.

Do this first labs · StackMaster Tech Participant Lab Guide

Before you begin

This guide collects the hands-on labs from the StackMaster Tech data-engineering program into one runnable workbook. Every lab uses the same **SkyOps Analytics** aircraft-fleet data, so set that up once here.

0.1 How to use this guide

Each lab has a **goal**, **prerequisites**, numbered **steps** (tick the box as you go), an **expected result**, and a green **checkpoint** telling you when you are done. Optional **stretch** tasks go further; the lined *notes* area is yours. Work top to bottom — later labs reuse earlier outputs.

Conventions

`inline code` is a command, table or setting. Dark blocks are code to run; the lighter “Console output” blocks show roughly what you should see. ☐ marks a step to complete.

0.2 Pick an environment

Option	Best for	How
A. Local	Tracks A & D, offline	<code>pip install pyspark jupyter</code> , run in a notebook
B. Databricks Free Edition	Tracks B & C (workspace, jobs)	Sign in via browser; a notebook + serverless compute
C. Skywise / Foundry	Client delivery	Code Workbook; same DataFrame API, platform-managed

Which to use

If you only have a laptop, **Option A** covers the PySpark and performance tracks. For the Databricks workspace, collaboration and Workflows labs (Track B and C), use **Option B** — it is free and browser-based.

0.3 Lab 0 — Generate the SkyOps dataset

Run this once to create the two core tables used throughout: `telemetry` (per-flight sensor readings) and the `aircraft` registry. It works locally and on Databricks.

```

from pyspark.sql import SparkSession
import pyspark.sql.functions as F

spark = (SparkSession.builder.appName("SkyOpsSetup")
        .master("local[*]") # omit on Databricks
        .getOrCreate())

types = ["A320", "A321", "A330", "A350", "B737", "B777"]
base = {"A320":2.4, "A321":2.7, "A330":5.9, "A350":5.4, "B737":2.5, "B777":7.1}

# Aircraft registry (small dimension – used for broadcast joins later)
registry = spark.createDataFrame(
    [(f"AC{1000+i}", types[i % len(types)], "SK" if i % 2 else "QF",
      f"VH-{chr(65+i%26)}{i:03d}", 150 + (i % 6) * 40) for i in range(60)],
    ["aircraft_id", "type", "airline", "tail_number", "seats"])

# Telemetry (large fact) – ~500k synthetic readings
tel = (spark.range(0, 500_000)
      .withColumn("aircraft_id", F.concat(F.lit("AC"),
                                          (1000 + (F.col("id") % 60)).cast("int").cast("string")))
      .withColumn("flight_id", (F.col("id") % 9000).cast("int"))
      .withColumn("ts", (F.lit(1_700_000_000) + F.col("id")).cast("timestamp"))
      .withColumn("altitude_ft", (35000 + (F.rand()*8000 - 4000)).cast("int"))
      .withColumn("engine_temp_c", (600 + F.rand()*120).cast("int"))
      .join(registry.select("aircraft_id", "type", "airline"), "aircraft_id", "left")
      .withColumn("fuel_tph", F.expr("element_at(map(" +
          ", ".join(f"'{k}', {v}'" for k, v in base.items()) + " ), type)"
          * (1 + (F.rand()-0.5)*0.12)))
      .withColumn("fuel_flow_kgph", F.col("fuel_tph")*1000))

registry.write.mode("overwrite").parquet("skyops/aircraft")
tel.write.mode("overwrite").parquet("skyops/telemetry")
print("telemetry rows:", tel.count(), "| registry rows:", registry.count())

```

CONSOLE OUTPUT

```
telemetry rows: 500000 | registry rows: 60
```

On Databricks: register as tables

After writing, register Unity Catalog tables so labs can use

`spark.table("skyops_prod.fleet.telemetry")`: create a catalog/schema and `CREATE TABLE ... USING parquet LOCATION 'skyops/telemetry'`, or simply read the Parquet paths directly in local mode.

Schema reference — keep this handy:

telemetry	aircraft (registry)
flight_id, aircraft_id, ts	aircraft_id, type, airline
airline, type, altitude_ft	tail_number, seats
fuel_tph, fuel_flow_kgph, engine_temp_c	

Setup checkpoint

You are ready for every lab once `skyops/telemetry` and `skyops/aircraft` exist and the row counts above print. Keep this notebook open — you will reuse the `spark` session throughout.



PySpark Fundamentals

Seven labs that take you from your first SparkSession to a complete streaming pipeline.

PySpark Fundamentals — Labs

These labs accompany the **PySpark Fundamentals** handbook. They build the core DataFrame, SQL, window and streaming skills on the SkyOps data. Local mode (Option A) is enough for all of them.

1.1

Hello, Spark

Track A · Module 1 · ~15 min

Goal. Create a SparkSession, build a small DataFrame, and confirm Spark is running.

Prerequisites

- ▶ Setup Lab 0 complete (a working `spark` session)

Steps

- ☐ 1 Open a Python shell or notebook and import `SparkSession`.
- ☐ 2 Build a tiny DataFrame of three aircraft with columns `type` and `fuel_tph`.
- ☐ 3 Add a column `fuel_flow_kgph = fuel_tph * 1000` with `withColumn`.
- ☐ 4 Call `df.show()` and confirm the new column appears.
- ☐ 5 Print `spark.version`.

Expected result

CONSOLE OUTPUT

```
+-----+-----+-----+
| type|fuel_tph|fuel_flow_kgph|
+-----+-----+-----+
| A320|      2.4|        2400.0|
| B777|      7.1|        7100.0|
+-----+-----+-----+
3.5.x
```

✓ Checkpoint — you are done when

`df.show()` prints your rows including `fuel_flow_kgph`, and `spark.version` prints a 3.x version.

➤ **Stretch.** Re-run with `df.printSchema()` and note the inferred types.

YOUR NOTES

3.1

Stand up your environment

Track A · Module 3 · ~20 min

Goal. Configure a real SparkSession, run a smoke test, and read the Spark Web UI.

Prerequisites

- ▶ A chosen environment (Option A, B or C)

Steps

- ☐ 1 Create a SparkSession with `appName("SkyOps")` and (local) `master("local[*]")`.
- ☐ 2 Run a smoke test: `spark.range(0, 1_000_000).filter("id % 7 = 0").count()`.
- ☐ 3 Print `spark.sparkContext.uiWebViewUrl` and open it (local: `http://localhost:4040`).
- ☐ 4 In the **Jobs** tab, find the job your `count()` created.
- ☐ 5 Note how many stages and tasks it ran.

✓ Checkpoint — you are done when

The count returns 142858, and you can see its job (with stages and tasks) in the Spark UI Jobs tab.

➤ **Stretch.** On Databricks/Skywise, locate the same UI via the cluster or build's Spark details instead of port 4040.

YOUR NOTES

5.1

SQL vs DataFrame parity

Track A · Module 5 · ~20 min

Goal. Prove the SQL and DataFrame APIs produce the same plan and result.

Prerequisites

- ▶ The telemetry Parquet from Lab 0

Steps

- ☐ 1 Read `skyops/telemetry` and register a temp view:
`df.createOrReplaceTempView("telemetry")`.
- ☐ 2 Compute average `altitude_ft` per `aircraft_id` with `spark.sql(...)`.
- ☐ 3 Compute the same thing with the DataFrame API (`groupBy().avg()`).
- ☐ 4 Confirm the two result sets match (sort and compare a few rows).
- ☐ 5 Run `.explain()` on both and compare the physical plans.

✓ Checkpoint — you are done when

Both queries return identical averages, and their `explain()` plans are essentially the same — Catalyst compiles both to the same physical plan.

➤ **Stretch.** Swap the aggregation to a windowed running average and compare plans again.

YOUR NOTES

6.1

Prove laziness

Track A · Module 6 · ~20 min

Goal. See lazy evaluation and caching first-hand in the Spark UI.

 Prerequisites

- ▶ A working session and the telemetry data

 Steps

- ☐ **1** Build a 4-step chain (read → filter → select → groupBy) **without** an action; confirm nothing runs yet.
- ☐ **2** Call `count()` and watch one job appear in the Jobs tab.
- ☐ **3** Call a second action (e.g. `collect()` a small result) and watch a **second** job appear.
- ☐ **4** Add `.cache()` and a warm-up `count()` to materialise it.
- ☐ **5** Run the second action again and compare its job duration to step 3.

✓ **Checkpoint — you are done when**

Two separate jobs appear for two actions, and the cached run's job is noticeably shorter than the uncached one.

➤ **Stretch.** Open the **Storage** tab and confirm the cached dataset shows 100% fraction cached.

YOUR NOTES

7.1

Top reading per aircraft

Track A · Module 7 · ~25 min

Goal. Use a window function plus a broadcast join to label the busiest reading per aircraft.

 Prerequisites

- ▶ telemetry and aircraft registry from Lab 0

 Steps

- ☐ **1** Define a window partitioned by `aircraft_id` ordered by `fuel_flow_kgph` descending.
- ☐ **2** Add `row_number()` over that window as column `rn`.
- ☐ **3** Filter to `rn == 1` to keep each aircraft's highest-fuel reading.
- ☐ **4** `broadcast()`-join the aircraft registry to add `tail_number` and `airline`.
- ☐ **5** Show the top 10 by fuel flow.

✓ Checkpoint — you are done when

You get exactly one row per aircraft (its peak fuel-flow reading), labelled with tail number and airline.

➤ **Stretch.** Swap `row_number()` for `rank()` and explain the difference on ties.

YOUR NOTES

8.1

Your first stream

Track A · Module 8 · ~25 min

Goal. Run a Structured Streaming query with windowing and a checkpoint.

Prerequisites

- ▶ A working Spark session (local is fine)

Steps

- ☐ **1** Start a `rate`-source stream and write it to console; watch micro-batches print.
- ☐ **2** Add a 30-second **windowed** aggregation (count per window).
- ☐ **3** Set `.option("checkpointLocation", "skyops/_chk/lab8")`.
- ☐ **4** Let it run a few batches, then stop the query.
- ☐ **5** Restart the query and confirm it **resumes from the checkpoint** rather than restarting.

✓ Checkpoint — you are done when

Batches print on a timer, the windowed counts update, and after a restart the query continues from where it left off.

➤ **Stretch.** Add a watermark and observe how late data is handled.

YOUR NOTES

9.1

End-to-end pipeline

Track A · Module 9 · ~40 min

Goal. Build a full parse → window → Delta streaming pipeline and verify the output.

Prerequisites

- ▶ Streaming basics from Lab 8.1

Steps

- ☐ **1** Use a local Kafka topic, or the `rate` source as a stand-in producer of JSON telemetry events.

- ☐ **2** Define the JSON schema and parse `value` into typed columns.
- ☐ **3** Add a **watermark** and a windowed aggregation of average fuel per aircraft.
- ☐ **4** Write the result as a **Delta** table with a checkpoint.
- ☐ **5** Query the Delta output as a *batch* DataFrame and confirm rows landed.

✓ **Checkpoint — you are done when**

The streaming job writes aggregated rows to Delta, and a batch read of that table returns those rows.

➤ **Stretch.** Trigger an overheat (`engine_temp_c > 650`) and route it to a separate alert sink; confirm it arrives.

YOUR NOTES



Databricks & Azure

Four labs across the workspace, collaboration, Azure architecture, jobs and Pandas analysis.

Databricks & Azure — Labs

These accompany the **Databricks & Azure Data Engineering** handbook. Labs B·1.1, B·3.1 need a Databricks workspace (Option B); B·2.1 is a design exercise; B·4.1 runs anywhere with Pandas.

1.1

Collaborative notebook dev & versioning

Track B · Module 1 · ~30 min · pairs

Goal. Practise real-time collaboration and Git version control in the workspace.

Prerequisites

- ▶ A Databricks workspace (Option B)
- ▶ A shared Git repo you both can access
- ▶ Work in pairs (A and B)

Steps

- ☐ 1 Create a **Git folder** from the shared repo and branch off `main`.
- ☐ 2 Partner A adds a cell reading `skyops_prod.fleet.telemetry` and leaves a **comment** asking B to add a filter.
- ☐ 3 Partner B edits the cell live, resolves the comment, commits as `.ipynb` and pushes.
- ☐ 4 Open a **pull request**, review each other's diff, and merge to `main`.
- ☐ 5 Open the notebook's **revision history** and compare it with the Git commit log.

✓ Checkpoint — you are done when

The change is merged via a reviewed PR, and you can see both the auto revision history and the Git commit log.

➤ **Stretch.** Decide and note: which record — revision history or Git log — would your team trust for an audit, and why?

YOUR NOTES

2.1

Design the SkyOps architecture

Track B · Module 2 · ~25 min · whiteboard

Goal. Sketch the end-to-end SkyOps platform and place each Azure service.

Prerequisites

- ▶ Module 2 of the handbook as reference
- ▶ A whiteboard or paper

Steps

- ☐ 1 Draw the **source** (Event Hubs) feeding the platform.
- ☐ 2 Draw the **lake** (ADLS Gen2) and the three medallion layers as Delta tables (Bronze/Silver/Gold).
- ☐ 3 Add the **compute** (Databricks) and show where secrets come from (**Key Vault**).
- ☐ 4 Add the **serving** layer (Power BI on Gold).
- ☐ 5 Mark one place you would add a **kappa** streaming path.

✓ Checkpoint — you are done when

You have a single diagram naming the source, lake, medallion layers, compute, secrets and serving — and one justified streaming insertion point.

➤ **Stretch.** In one sentence, justify the streaming path you marked (what freshness requirement does it serve?).

YOUR NOTES

3.1

Schedule the SkyOps pipeline

Track B · Module 3 · ~30 min

Goal. Build and operate a scheduled, multi-task Databricks job.

Prerequisites

- ▶ A Databricks workspace (Option B)
- ▶ Two simple notebooks: ingest and clean

Steps

- ☐ 1 Under **Jobs & Pipelines**, create job `skyops_nightly_etl`.
- ☐ 2 Add two notebook tasks; make `clean_silver` depend on `ingest_bronze`.
- ☐ 3 Set **job compute**, a **2 AM** schedule, **2 retries**, a **30-min timeout** and an **email alert** on failure.
- ☐ 4 Run the job once, then open the runs list and the task **DAG**.
- ☐ 5 Force the second task to fail, then use **Repair run** to re-run only that task.

✓ Checkpoint — you are done when

The job runs as a 2-task DAG on a schedule, and Repair run re-executes only the failed task — not the successful upstream one.

➤ **Stretch.** Change the trigger from a schedule to a file-arrival trigger and describe when each is preferable.

YOUR NOTES

4.1

Explore the fleet extract

Track B · Module 4 · ~30 min

Goal. Profile, summarise, analyse and visualise the SkyOps extract with Pandas.

 Prerequisites

- ▶ Python with pandas, numpy, matplotlib, seaborn
- ▶ A Parquet extract (use skyops/telemetry, or export a Silver sample)

 Steps

- ☐ **1** Load the extract into a Pandas DataFrame.
- ☐ **2** Print `describe()` and the `type` value counts.
- ☐ **3** Build a grouped summary of average `fuel_tph` and p95 `altitude_ft` by aircraft type.
- ☐ **4** Using NumPy, compute the correlation between altitude and fuel.
- ☐ **5** Plot one **distribution** chart and one **comparison** chart; write a one-sentence finding under each.

 Expected result

CONSOLE OUTPUT

```
      flights  avg_fuel  p95_alt
type
B777      ...      7.10  41980.0
A320      ...      2.40  40990.0
```

 Checkpoint — you are done when

You have summary statistics, a grouped table, a correlation value, and two charts — each with a written insight.

➤ **Stretch.** Recompute the grouped summary by an `alt_band` column (low/cruise/high) using `pd.cut`.

YOUR NOTES



Azure Databricks Workflows

One end-to-end lab: deploy a workload through the UI and as code.

1 labs · StackMaster Tech Participant Lab Guide

Azure Databricks Workflows — Lab

This accompanies the **Azure Databricks Workflows** handbook. It compares the UI and as-code (Declarative Automation Bundle) deployment paths. Use a Databricks workspace (Option B).

C

Deploy the SkyOps nightly workload

Track C · ~40 min

Goal. Deploy the same job two ways — through the UI and as a versioned bundle — and recover a failure.

Prerequisites

- ▶ A Databricks workspace (Option B)
- ▶ The Databricks CLI installed and authenticated
- ▶ Two notebooks: ingest and clean

Steps

- ☐ 1 In the UI, create `skyops_nightly_etl` with two notebook tasks (`clean_silver` depends on `ingest_bronze`).
- ☐ 2 Set **serverless** compute, a **2 AM** schedule, **2 retries** and an **email alert**; run it once and open the run's DAG.
- ☐ 3 Now express the same job as a `databricks.yml` **bundle** with `dev` and `prod` targets.
- ☐ 4 Run `databricks bundle validate`, then `databricks bundle deploy -t dev`.
- ☐ 5 Trigger a run with `databricks bundle run skyops_nightly_etl -t dev`.
- ☐ 6 Force the second task to fail, then use **Repair run** to re-run only that task.

Expected result

CONSOLE OUTPUT

```
Validation OK!
Deploying resources...
Deployment complete!
  job skyops_nightly_etl  ->  ../jobs/1043
```

✓ Checkpoint — you are done when

The job exists and runs from both the UI and the deployed bundle, and Repair run recovers the partial failure.

➤ **Stretch.** Note which path — UI or bundle — you would trust for a production release, and list two reasons.

YOUR NOTES



Spark Performance

Six labs that diagnose and fix real bottlenecks, measured before and after in the Spark UI.

Spark Performance — Labs

These accompany the **Spark Performance Tuning** handbook. They form one connected story: Lab D·2.1 *diagnoses* a skewed join that Lab D·4.1 then *fixes* — always measuring in the Spark UI.

1.1

Read your own plan

Track D · Module 1 · ~20 min

Goal. Read a physical plan and locate the shuffles before tuning anything.

Prerequisites

- telemetry + aircraft registry from Lab 0

Steps

- ☐ **1** Write a query that joins `telemetry` to the aircraft registry and groups by `type`.
- ☐ **2** Run `explain(mode="formatted")`.
- ☐ **3** Mark every **Exchange** node (each is a shuffle / stage boundary).
- ☐ **4** Confirm the scan shows **PushedFilters** and only the needed columns.
- ☐ **5** Predict which stage will be the most expensive — and keep this plan.

✓ Checkpoint — you are done when

You can point to each shuffle in the plan and have a written prediction of the costliest stage. Save it for Lab D·4.1.

YOUR NOTES

2.1

Find the slow stage

Track D · Module 2 · ~25 min

Goal. Use the Spark UI to baseline a job and name its bottleneck.

Prerequisites

- The query from Lab D·1.1, run on the full telemetry

Steps

- ☐ **1** Run the query so it actually executes (add a `count()` or write).
- ☐ **2** Open the Spark UI and find the **job**, then its **slowest stage**.
- ☐ **3** Capture that stage's task **min / median / max** for duration, shuffle read and spill.
- ☐ **4** Write one sentence naming the bottleneck **signature** (skew? spill? heavy shuffle?).

- ☐ 5 State which later module fixes it.

Expected result

CONSOLE OUTPUT

```
Stage 7 (join)  duration  max 38s  median 0.6s
                shuffle read  max 1.9GB  median 12MB
                spill (memory) max 2.4GB
```

✓ Checkpoint — you are done when

You have a baseline: the slow stage's min/median/max captured, and a named bottleneck (you should see skew: max >> median).

➤ **Stretch.** Note the percentage of total stage time spent in the single slowest task.

YOUR NOTES

3.1

Measure a cache

Track D · Module 3 · ~20 min

Goal. Quantify the benefit of caching a reused DataFrame.

Prerequisites

- ▶ A cleaned DataFrame that feeds two aggregations

Steps

- ☐ 1 Run the two aggregations **uncached**; note total time and the repeated scan in the Spark UI.
- ☐ 2 Add `persist(StorageLevel.MEMORY_AND_DISK)` and an action to materialise it.
- ☐ 3 Re-run the two aggregations and compare the total time.
- ☐ 4 Open the **Storage** tab and check the **fraction cached**.
- ☐ 5 Call `unpersist()` when done.

✓ Checkpoint — you are done when

The cached run is faster, the repeated scan disappears, and the Storage tab shows the dataset cached (ideally 100%).

➤ **Stretch.** Try `MEMORY_AND_DISK_SER` and note the memory vs CPU trade-off.

YOUR NOTES

4.1

Halve a query's runtime

Track D · Module 4 · ~30 min

Goal. Fix the skewed join from Lab D·2.1 with a broadcast and measure the win.

Prerequisites

- ▶ The skewed join and baseline from Lab D·2.1

Steps

- ☐ 1 Confirm the join is a **SortMergeJoin** in `explain()`.
- ☐ 2 Wrap the small (registry) side in `broadcast()`.
- ☐ 3 Re-run `explain()` and confirm it is now a **BroadcastHashJoin**.
- ☐ 4 Re-run the query and compare the stage's **max-task time** with your Lab D·2.1 baseline.
- ☐ 5 Experiment with `spark.sql.shuffle.partitions` and watch AQE coalesce in the **SQL** tab.

Expected result

CONSOLE OUTPUT

```
before: Stage max-task 38 s (SortMergeJoin)
after:  Stage max-task  2.7 s (BroadcastHashJoin)
```

✓ Checkpoint — you are done when

The join becomes a BroadcastHashJoin, the giant single-task shuffle disappears, and the slow stage drops dramatically (e.g. 38s → under 3s).

➤ **Stretch.** Raise `autoBroadcastJoinThreshold` and confirm Spark auto-broadcasts without the hint.

YOUR NOTES

5.1

Cure a spill

Track D · Module 5 · ~25 min

Goal. Eliminate disk spill by right-sizing partitions.

Prerequisites

- ▶ A stage in the Spark UI that reports non-zero spill

Steps

- ☐ 1 Find a stage whose tasks report **Spill (memory/disk)**; note its partition count and spill size.
- ☐ 2 Increase `spark.sql.shuffle.partitions` (or `repartition`) so each partition is smaller.
- ☐ 3 Re-run and confirm spill drops toward zero and the stage speeds up.
- ☐ 4 If spill persists, decide whether **skew** (Module 4) or executor memory is the real cause.
- ☐ 5 Record the partition count that removed the spill.

✓ Checkpoint — you are done when

The stage's reported spill falls to (near) zero and its duration improves after partitions are sized smaller.

➤ **Stretch.** Compute a target partition count for ~128–256 MB per partition from the stage's shuffle-read size.

YOUR NOTES

6.1

Replace a slow UDF

Track D · Module 6 · ~30 min

Goal. Swap a row-by-row Python UDF for a vectorised alternative and measure the speed-up.

Prerequisites

- ▶ A transform using a plain Python UDF (write one if needed)

Steps

- ☐ **1** Time the transform and confirm in the Spark UI that the stage is **Python-bound**.
- ☐ **2** Try to replace the UDF with a **built-in** expression (`when` / `otherwise` , math, string ops).
- ☐ **3** If custom Python is unavoidable, rewrite it as a vectorised `@pandas_udf` .
- ☐ **4** Ensure Arrow is enabled (`spark.sql.execution.arrow.pyspark.enabled=true`).
- ☐ **5** Re-run and compare the stage time; record the speed-up.

Expected result

CONSOLE OUTPUT

```
plain Python UDF stage: 41.0 s
pandas_udf (Arrow) stage: 0.9 s
```

✓ Checkpoint — you are done when

The built-in or Pandas-UDF version runs dramatically faster than the plain Python UDF (often 10–100×).

➤ **Stretch.** Apply a single-node scikit-learn model across the data inside a Pandas UDF for parallel inference.

YOUR NOTES

APPENDIX

Completion checklist

Tick each lab as you finish it. By the end you will have a working SkyOps pipeline and a measured, before-and-after performance story.

- ☐ **0** Setup — generate the SkyOps dataset
- ☐ **A.1.1** Hello, Spark
- ☐ **A.3.1** Stand up your environment
- ☐ **A.5.1** SQL vs DataFrame parity
- ☐ **A.6.1** Prove laziness
- ☐ **A.7.1** Top reading per aircraft
- ☐ **A.8.1** Your first stream
- ☐ **A.9.1** End-to-end pipeline
- ☐ **B.1.1** Collaborative notebook dev & versioning
- ☐ **B.2.1** Design the SkyOps architecture

- ☐ **B.3.1** Schedule the SkyOps pipeline
- ☐ **B.4.1** Explore the fleet extract
- ☐ **C** Deploy the SkyOps nightly workload
- ☐ **D.1.1** Read your own plan
- ☐ **D.2.1** Find the slow stage
- ☐ **D.3.1** Measure a cache
- ☐ **D.4.1** Halve a query's runtime
- ☐ **D.5.1** Cure a spill
- ☐ **D.6.1** Replace a slow UDF

Troubleshooting quick reference

Symptom	Likely fix
<code>No module named pyspark</code>	<code>pip install pyspark</code> ; check the right kernel/venv
Spark UI not at :4040	Another app holds 4040 — try :4041; on Databricks/Skywise open it via the cluster/build
Job/stage very slow on one task	Data skew — broadcast the small side or rely on AQE skew-join (Lab D·4.1)
Spill or OutOfMemory	Partitions too big — raise <code>shuffle.partitions</code> / repartition (Lab D·5.1)
broadcast() ignored	Table above the threshold or stats missing — raise <code>autoBroadcastJoinThreshold</code>
UDF stage dominates time	Replace with a built-in or vectorised <code>@pandas_udf</code> (Lab D·6.1)
Stream will not resume	Reuse the same <code>checkpointLocation</code> ; do not delete it between runs
Cache seems to do nothing	Materialise it with an action; check the Storage tab's fraction cached



Golden rule of every performance lab

Always **measure before and after** in the Spark UI, and change **one thing at a time**. A fix you cannot see in the numbers is not a fix.